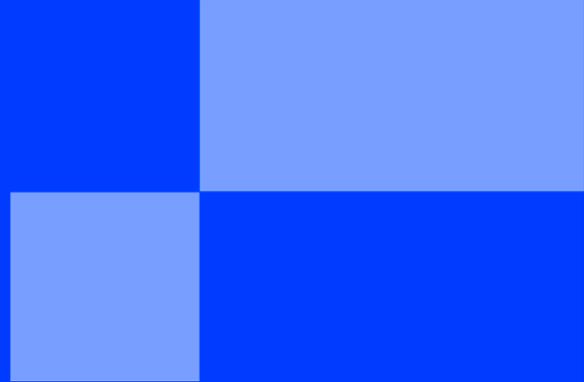




INFOGRAPHIE

< WINTER CAMP 2026 />

INFOGRAPHIE

A faire dans un nouveau notebook!

Prérequis

Avoir fait **l'Atelier Starter - Introduction à l'Analyse de Données**.

Contexte

Générer des fiches de statistiques Pokémon.

Importer les bibliothèques

Nous avons besoin de `pandas` et de `random`.

Expérimentation:

- ✓ Dans un nouveau bloc de code:
 - Importer la bibliothèque `pandas`
 - Importer la bibliothèque `random`



✓ N'oubliez pas d'exécuter vos cellules de code

Lire le CSV et les stocker dans un DataFrame

Ici le CSV à lire est `complete_pokedex.csv`

Expérimentation:

- ✓ Dans un nouveau bloc de code:
 - Lisez le csv `complete_pokedex.csv` et stockez-le dans un dataframe `pokemon_df`



- ✓ Comme précédemment, prenez le temps d'observer le dataframe: nombre de lignes, nom des colonnes, etc.

Sélectionner un Pokémon de façon aléatoire

Les Pokémon sont numérotés de 1 à 1024. Ces numéros correspondent à la colonne `id` du DataFrame `pokemon_df`.

Nous allons ici utiliser la fonction `random.randint()` de Python pour choisir un nombre aléatoire correspondant au numéro du Pokémon dont on va devoir générer la fiche de statistique.

Exemple:

Pour générer un nombre aléatoire allant de 1 à 6 (jet de dé) en Python:

```
...  
>>> random.randint(1, 6)  
4  
>>> random.randint(1, 6)  
1
```

Expérimentation:

- ✓ Dans un nouveau bloc de code:
 - Générez un nombre aléatoire entre 1 et 1024, stockez ce nombre dans une variable `random_id`
 - Récupérez la ligne dont la colonne `id` vaut `random_id` dans le DataFrame `pokemon_df` (filtre), stockez cette ligne dans une variable `my_pokemon`.



- ✓ Attention ne confondez pas la valeur de `id` dans `pokemon_df` (commence à 1) et le numéro de la ligne dans `pokemon_df` (commence à 0)
- ✓ Vous pouvez afficher le contenu de vos variables (ici `my_pokemon`) en terminant votre cellule de code par une ligne avec cette variable.

Récupérer les informations sur notre Pokémon

Afin de faire notre fiche, nous avons besoin de récupérer plusieurs informations sur notre Pokémon.

- ✓ Nom du Pokémon
- ✓ Ses types (2 types par Pokémon)
- ✓ Ses statistiques: représentant ses performances globales au combat
 - de PV, d'attaque, de défense, d'attaque spéciale, de défense spéciale, de vitesse
- ✓ Son indice de capture: valeur qui plus elle est élevée, plus le pokémon sera facile à capturer (maximum 255)

Comment récupérer la valeur d'une colonne depuis un DataFrame contenant qu'une seule ligne ?

nom	age	classe	moyenne
Bob	16	1ère	14.2

```
df.nom.values[0]  
Bob
```

```
df.moyenne.values[0]  
14.2
```

Nous pouvons stocker ces valeurs dans des variables afin de les réutiliser par la suite.

```
>>> nom_eleve = df.nom.values[0]
```

```
>>> moyenne_eleve = df.moyenne.values[0]
```

```
>>> nom_eleve  
Bob
```

```
>>> moyenne_eleve + 1
```

Expérimentation:

- ✓ Dans un nouveau bloc de code:
 - Dans une variable `my_poke_name` récupérez le nom de votre Pokémon
 - Dans une variable `my_poke_catch` récupérez l'indice de capture de votre Pokémon
 - Dans une variable `my_poke_type1` récupérez le premier type de votre Pokémon
 - Dans une variable `my_poke_type2` récupérez le second type de votre Pokémon
 - Dans une variable `my_poke_hp` récupérez la valeur de sa statistique de PV
 - Dans une variable `my_poke_atk` récupérez la valeur de sa statistique d'attaque
 - Dans une variable `my_poke_def` récupérez la valeur de sa statistique de défense
 - Dans une variable `my_poke_spe_atk` récupérez la valeur de sa statistique d'attaque spéciale
 - Dans une variable `my_poke_spe_def` récupérez la valeur de sa statistique de défense spéciale
 - Dans une variable `my_poke_speed` récupérez la valeur de sa statistique de vitesse
 - Dans une variable `my_poke_stats_total` stockez la somme totale des statistiques de votre Pokémon: `my_poke_hp + my_poke_atk + my_poke_def + my_poke_spe_atk + my_poke_spe_def + my_poke_speed`



- ✓ Vous pouvez faire un `print(my_poke_name, my_poke_type1, my_poke_type2, ..., my_poke_speed)` à la fin de votre bloc de code pour vérifier la valeur de toutes vos variables

Récupérer des informations supplémentaires sur notre Pokémon

Il est possible de générer une URL pointant sur une illustration de votre Pokémon et de ses types.

Pour l'illustration d'un Pokemon, l'URL est sous la forme suivante :

```
https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/<id_du_pokemon>.png
```

où "id_du_pokemon" correspond au numéro d'un Pokémon.

Par exemple:

- En allant sur <https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/25.png> vous verrez le sprite de Pikachu.

- ✓ Pour l'illustration d'un type, l'URL est sous la forme suivante :

```
https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/types/generation-ix/scarlet-violet/<id_du_type>.png
```

Par exemple:

- En allant sur <https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/types/generation-ix/scarlet-violet/10.png> vous verrez une illustration pour le type Feu.
- Sur <https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/types/generation-ix/scarlet-violet/13.png> vous verrez une illustration pour le type Electrique.

Pour cela nous allons déclarer et utiliser des fonctions:

Une fonction est un bloc de code écrit une seule fois et auquel un nom est donné pour pouvoir la réutiliser facilement. Elle sert principalement à ranger un code complexe sous une forme plus facile à utiliser.

Nous allons déclarer une fonction `get_img_url(id)` (récupère l'URL pour l'illustration d'un Pokemon ayant pour numéro: `id`)

```
def get_img_url(id):  
    return f"https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/{id}.png"
```

Vous pouvez directement copier-coller cette fonction dans un bloc de code dans votre notebook Jupyter

Déclarez également la fonction `get_type_url(name)` (récupère l'URL pour l'illustration d'un type ayant pour dénomination `name`)

```
def get_type_url(name):  
    types = {  
        'normal': 1,  
        'fighting': 2,  
        'flying': 3,  
        'poison': 4,  
        'ground': 5,  
        'rock': 6,  
        'bug': 7,  
        'ghost': 8,  
        'steel': 9,  
        'fire': 10,  
        'water': 11,  
        'grass': 12,  
        'electric': 13,  
        'psychic': 14,  
        'ice': 15,  
        'dragon': 16,  
        'dark': 17,  
        'fairy': 18  
    }  
    return f"https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/types/generation-ix/scarlet-violet/{types[name]}.png"
```

Vous pouvez directement copier-coller cette fonction dans un bloc de code dans votre notebook Jupyter

Pour utiliser ces fonctions vous pouvez entrer dans un bloc de code:

```
>>> get_img_url(6)
'https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/6.png'
```

Donne un lien vers l'image d'un Dracaufeu.

ou

```
>>> get_img_url(random_id)
'https://raw.githubusercontent.com/...'
```

Donne un lien vers l'image de votre Pokémon choisi aléatoirement en début d'atelier.

```
>>> get_type_url("water")
'https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/types/generation-ix/scarlet-violet/11.png'
```

Donne un lien vers l'illustration du type Eau.

```
>>> get_type_url(my_poke_type1)
'https://raw.githubusercontent.com/...'
```

Donne un lien vers l'illustration du premier type de votre Pokémon choisi aléatoirement en début d'atelier.

Expérimentation:

- ✓ Dans un nouveau bloc de code:
 - En utilisant la fonction `get_img_url()`, récupérez le lien vers le sprite de votre Pokémon dans une variable `my_poke_img`
 - En utilisant (deux fois) la fonction `get_type_url()`, récupérez le lien vers les illustrations des deux types de votre Pokémon dans deux variables nommées respectivement `my_poke_type1_url` et `my_poke_type2_url`



- ✓ Vous pouvez ici aussi utiliser `print(my_poke_img, my_poke_type1_url, my_poke_type2_url)` afin de vérifier si vos variables contiennent les bonnes valeurs.
- ✓ Certains Pokémon n'ont qu'un seul type, dans ce cas il est normal que `my_poke_type1_url` et `my_poke_type2_url` aient la même valeur.

Représenter la donnée du taux de capture

L'indice de capture fourni dans le fichier CSV est une valeur entre 0 et 255 indiquant s'il est plus ou moins facile à capturer.

Cette valeur n'est pas très parlante pour le commun des mortels.

Nous allons donc la remplacer par un pourcentage: la probabilité d'attraper le Pokémon en lui lançant une PokéBall.

Exemple:

- Une valeur de 50% signifie qu'on a une chance sur deux d'attraper un Pokémon en lui lançant une PokéBall.
- Une valeur de 95% signifie qu'on est presque garanti d'attraper un Pokémon en lui lançant une PokéBall.
- Une valeur de 15% signifie que le Pokémon sera plutôt difficile à attraper, il faudra lui lancer en moyenne 6,6 PokéBalls pour le capturer (même si très rarement une seule suffira, ou au contraire il faudra lui en lancer beaucoup plus).

La formule pour convertir l'indice de capture (0-255) en pourcentage de chance d'attraper le Pokémon est la suivante:

$$100 \times \left(\left(\frac{\min \left(\frac{C \times B}{3}, 255 \right)}{255} \right)^{\frac{3}{16}} \right)^4$$

où

C = indice de capture du Pokémon

B = coefficient de capture de la Ball

Le coefficient de capture de la Ball change en fonction du type de Ball utilisée:

$$B = PokBall = 1$$

$$B = SuperBall = 1.5$$

$$B = HyperBall = 2$$

Utiliser une HyperBall est plus efficace que d'utiliser une SuperBall qui est elle-même plus efficace que d'utiliser une PokeBall.

Cette formule peut sembler effrayante, nous allons donc nous servir d'une fonction afin de pouvoir l'utiliser facilement et cacher sa complexité.

```
def catch_pct(C, B):  
    return round(100.0 * (((min((C * B) / 3.0, 255.0) / 255.0) ** (3.0 / 16.0)) ** 4)  
                ,2)
```

Vous pouvez directement copier-coller cette fonction dans un nouveau bloc de code de votre notebook Jupyter.

Exemples:

Prenons un Pokemon avec un indice de capture de 190.

✓ La probabilité de l'attraper en lui lançant une PokéBall:

```
>>> catch_pct(190, 1)  
35.2
```

✓ Une SuperBall:

```
>>> catch_pct(190, 1.5)  
47.7
```

Avec un Pokémon avec un indice de capture de 25 (donc plus difficile à attraper).

✓ Avec une PokéBall:

```
>>> catch_pct(25, 1)  
7.7
```

✓ Une HyperBall:

```
>>> catch_pct(25, 2)  
12.9
```

Expérimentation:

- ✓ Dans un nouveau bloc de code:
 - Utilisez (plusieurs fois) la fonction `catch_pct()` pour calculer et stocker dans les variables `proba_poke`, `proba_super`, `proba_hyper` respectivement:
 - * Le pourcentage de chance d'attraper votre Pokémon choisi aléatoirement en début d'atelier avec une PokéBall
 - * une SuperBall
 - * une HyperBall



- ✓ Lors des étapes précédentes vous avez normalement récupéré l'indice de capture de votre Pokémon dans une variable appelée `my_poke_catch`.

Rendu de la fiche de statistique

Lorsque vous avez terminé toutes les étapes précédentes, vous pouvez enfin afficher le rendu de votre fiche dans votre notebook Jupyter avec le code suivant (à copier-coller directement dans un nouveau bloc de code à la fin de votre notebook).

```
from IPython.display import HTML, display

html_code = f"""
<!DOCTYPE html>
<html lang="fr">
<head>
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=
    swap" rel="stylesheet">
  <style>
    :root {{
      --bg-color: transparent;
      --card-bg: #ffffff;
      --text-main: #2c3e50;
      --text-secondary: #7f8c8d;
      --accent-border: #dfe6e9;
      --main-color: #58A8F8;

      --hp-color: #FF5959;
      --atk-color: #F5AC78;
      --def-color: #FAE078;
      --spatk-color: #9DB7F5;
      --spdef-color: #A7DB8D;
      --spd-color: #FA92B2;
    }}

    .pokemon-card {{
      font-family: 'Poppins', sans-serif;
      background: var(--card-bg);
      width: 320px;
      padding: 30px 20px;
    }}
  </style>
</head>
</html>
"""

display(HTML(html_code))
```

```

        border-radius: 24px;
        box-shadow: 0 10px 30px rgba(0,0,0,0.1);
        text-align: center;
        position: relative;
        margin: 20px auto;
        overflow: hidden;
    }}

    .image-container {{
        width: 140px; height: 140px;
        margin: 0 auto 10px;
        position: relative;
        display: flex; justify-content: center; align-items: center;
    }}

    .image-bg {{
        position: absolute; width: 100%; height: 100%;
        background: radial-gradient(circle, rgba(88, 168, 248, 0.2) 0%, transparent
            70%);
        border-radius: 50%; z-index: 0;
    }}

    .image-container img {{
        width: 100%; height: 100%;
        object-fit: contain; position: relative; z-index: 1;
        filter: drop-shadow(0 5px 5px rgba(0,0,0,0.2));
        image-rendering: pixelated;
    }}

    .pokemon-name {{
        font-size: 1.8rem; font-weight: 700; color: var(--text-main);
        margin: 5px 0 5px 0; text-transform: capitalize;
    }}

    .name-underline {{
        height: 4px; width: 40px; background: var(--main-color);
        margin: 0 auto 20px; border-radius: 2px; opacity: 0.8;
    }}

    .types-container {{
        display: flex; justify-content: center; gap: 12px;
        margin-bottom: 25px; align-items: center;
    }}

    .type-img-wrapper img {{
        height: 28px; width: auto;
        filter: drop-shadow(0 2px 3px rgba(0,0,0,0.1));
        transition: transform 0.2s;
    }}

    .type-img-wrapper img:hover {{ transform: scale(1.1); }}

    .stats-box {{
        position: relative;
        border: 1px solid var(--accent-border);
        border-radius: 18px;
        padding: 25px 15px 15px;
        margin-bottom: 25px;
        text-align: left;
        background: linear-gradient(to bottom, #ffffff, #fafbfc);
    }}

```

```

    .total-label {{
      position: absolute; top: -12px; right: 15px;
      background: var(--text-main); color: white;
      padding: 4px 10px; font-weight: 700; font-size: 0.75rem;
      border-radius: 10px; text-transform: uppercase;
    }}

    .stat-row {{ display: flex; align-items: center; margin-bottom: 10px; }}
    .stat-name {{ width: 70px; font-weight: 600; color: var(--text-secondary); font-size: 0.75rem; }}
    .stat-bar-bg {{ flex-grow: 1; height: 6px; background-color: #eff2f6; border-radius: 4px; overflow: hidden; margin: 0 10px; }}
    .stat-bar-fill {{ height: 100%; border-radius: 4px; }}
    .stat-value {{ width: 25px; text-align: right; font-weight: 700; color: var(--text-main); font-size: 0.8rem; }}

    .catch-container {{ display: flex; justify-content: space-between; padding: 0 5px; }}
    .catch-circle {{
      width: 70px; height: 80px;
      border-radius: 16px; background: white;
      border: 1px solid var(--accent-border);
      display: flex; flex-direction: column; justify-content: center; align-items: center;
      transition: all 0.3s ease;
    }}
    .catch-ball-img {{ width: 32px; margin-bottom: 4px; }}
    .catch-percent {{ font-size: 0.85rem; font-weight: 700; color: var(--text-main); }}
    .catch-label {{ font-size: 0.6rem; color: var(--text-secondary); }}
    .catch-circle:hover {{ transform: translateY(-3px); border-color: var(--main-color); }}

  </style>
</head>
<body>

  <div class="pokemon-card">

    <div class="image-container">
      <div class="image-bg"></div>
      
    </div>

    <div class="pokemon-name">{my_poke_name}</div>
    <div class="name-underline"></div>

    <div class="types-container">
      <div class="type-img-wrapper"></div>
      {f'<div class="type-img-wrapper"></div>' if
        my_poke_type1_url != my_poke_type2_url else ''}
    </div>

    <div class="stats-box">
      <div class="total-label">Total : {my_poke_stats_total}</div>

      <div class="stat-row">
        <span class="stat-name">PV</span>
        <div class="stat-bar-bg"><div class="stat-bar-fill" style="width: {(
          my_poke_hp / 255) * 100}%"; background-color: var(--hp-color);"></div></div>
      </div>
    </div>
  </div>
</body>
</html>

```

```

        <span class="stat-value">{my_poke_hp}</span>
    </div>
    <div class="stat-row">
        <span class="stat-name">Attaque</span>
        <div class="stat-bar-bg"><div class="stat-bar-fill" style="width: {(
            my_poke_atk / 255) * 100}%; background-color: var(--atk-color);"></div>
        </div>
        <span class="stat-value">{my_poke_atk}</span>
    </div>
    <div class="stat-row">
        <span class="stat-name">Défense</span>
        <div class="stat-bar-bg"><div class="stat-bar-fill" style="width: {(
            my_poke_def / 255) * 100}%; background-color: var(--def-color);"></div>
        </div>
        <span class="stat-value">{my_poke_def}</span>
    </div>
    <div class="stat-row">
        <span class="stat-name">Atq. Spé.</span>
        <div class="stat-bar-bg"><div class="stat-bar-fill" style="width: {(
            my_poke_spe_atk / 255) * 100}%; background-color: var(--spatk-color)
            ;"></div></div>
        <span class="stat-value">{my_poke_spe_atk}</span>
    </div>
    <div class="stat-row">
        <span class="stat-name">Déf. Spé.</span>
        <div class="stat-bar-bg"><div class="stat-bar-fill" style="width: {(
            my_poke_spe_def / 255) * 100}%; background-color: var(--spdef-color)
            ;"></div></div>
        <span class="stat-value">{my_poke_spe_def}</span>
    </div>
    <div class="stat-row">
        <span class="stat-name">Vitesse</span>
        <div class="stat-bar-bg"><div class="stat-bar-fill" style="width: {(
            my_poke_speed/255) * 100}%; background-color: var(--spd-color);"></div>
        </div>
        <span class="stat-value">{my_poke_speed}</span>
    </div>
</div>

<div class="catch-container">
    <div class="catch-circle">
        
        <span class="catch-percent">{proba_poke}%</span>
        <span class="catch-label">Poke Ball</span>
    </div>

    <div class="catch-circle">
        
        <span class="catch-percent">{proba_super}%</span>
        <span class="catch-label">Super Ball</span>
    </div>

    <div class="catch-circle">
        
        <span class="catch-percent">{proba_hyper}%</span>
        <span class="catch-label">Hyper Ball</span>
    </div>
</div>

```

```
</div>
</body>
</html>
"""

display(HTML(html_code))
```

Félicitations, vous voyez normalement la fiche de statistique de votre Pokémon complétée.

Si ce n'est pas le cas : vérifiez que vous avez bien nommé toutes les variables comme demandé.

Idées de bonus:

Voici quelques idées pour aller plus loin avec votre fiche de statistiques Pokémon :

Générer plusieurs fiches

Générez des fiches pour plusieurs Pokémon différents. Vous pouvez relancer toutes les cellules de votre notebook avec un numéro de Pokémon que vous avez choisi au lieu d'un numéro aléatoire.

Récupérer les sprite en version chromatique

Vous pouvez rendre votre fiche encore plus intéressante en affichant le sprite de votre Pokémon dans sa version chromatique (shiny), qui propose des couleurs alternatives souvent recherchées par les collectionneurs.

Pour récupérer l'URL du sprite chromatique, il suffit de remplacer `pokemon` par `pokemon/shiny` dans l'URL utilisée précédemment. Par exemple, l'URL pour le Léviator chromatique (id 130) devient :

```
https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/shiny/130.png
```

au lieu de

```
https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/130.png
```

Vous pouvez donc créer une nouvelle fonction Python pour obtenir l'URL de la version chromatique :

```
def get_shiny_img_url(id):
    return f"https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/
```

```
shiny/{id}.png"
```

Utilisez cette fonction pour afficher l'image shiny dans votre fiche, à côté ou en dessous du sprite classique.

Personnaliser le design

Modifiez les couleurs, les styles CSS, ou la mise en page de votre fiche pour créer un design unique. Par exemple :

- Changez les couleurs des barres de statistiques

Comparer deux Pokémons

Créez une fiche de comparaison qui affiche côte à côte les statistiques de deux Pokémons différents.

Ajouter des informations supplémentaires

Enrichissez votre fiche avec d'autres informations disponibles dans le DataFrame, comme :

- La génération à laquelle il appartient
- Son habitat
- Sa description
- Le cri du Pokémon https://raw.githubusercontent.com/PokeAPI/cries/main/cries/pokemon/latest/<id_du_pokemon>.ogg
- etc.

v1

{EPITECH}